

國立清華大學
National Tsing Hua University

碩士論文
Master's Thesis

這裡填入論文中文題名
Type English Title Here

系所別：資訊安全研究所

Dept. / Grad. Inst.: Institute of Information Security

學號 Student ID: 111164529

研究生 Author：黃宏勝 Hong-Sheng Huang

指導教授 Advisor：

中華民國一一五年七月

July 2026

國立清華大學
National Tsing Hua University

碩士論文
Master's Thesis

這裡填入論文中文題名
Type English Title Here

系所別：資訊安全研究所

Dept. / Grad. Inst.: Institute of Information Security

學號 Student ID: 111164529

研究生 Author：黃宏勝 Hong-Sheng Huang

指導教授 Advisor：

中華民國一一五年七月

July 2026

摘要

摘要是整篇論文的濃縮總結，需精簡扼要，涵蓋論文核心內容，邏輯連貫地總結各章節要點，吸引讀者興趣。建議控制在一頁內，約400–600字，避免使用圖表。

內容要素：

- 研究動機與目標：闡述研究背景、核心問題及主要目標。
- 研究範圍：明確界定研究的課題與範疇。
- 研究方法：概述方法、技術或工具，並簡述其合理性。
- 主要結果：突出核心發現或數據。
- 結果比較與意義：將結果與研究目標及文獻比較，凸顯貢獻。
- 未來影響與建議：說明研究對學術或實務的助益，提出未來方向。

撰寫建議：

- 使用精煉語言，確保非領域專家也能理解。
- 完成論文其他部分後再提煉摘要，確保內容全面。
- 使用純文字格式（無HTML或特殊格式），便於圖書館建檔與網路檢索。
- 附上5–7個關鍵詞，提升論文檢索率。

關鍵字：論文格式、圖書館建檔、範例、提煉、圖書館建檔

Abstract

The abstract is a concise summary of the dissertation, encapsulating its core content in a clear, logically coherent manner to engage readers. It should be limited to one page, approximately 300–500 words, and avoid including figures or tables.

Content Elements

- **Research Motivation and Objectives:** Describe the background, core research question, and primary goals.
- **Research Scope:** Clearly define the topics and boundaries of the study.
- **Methodology:** Summarize the methods, techniques, or tools used and briefly justify their selection.
- **Key Results:** Highlight the most significant findings or data.
- **Comparison and Significance:** Compare results with the initial objectives and existing literature, emphasizing contributions.
- **Impact and Future Directions:** Discuss the study's benefits to academic or practical fields and suggest future research or applications.

Writing Tips

- Use precise, accessible language to ensure clarity for non-experts.
- Draft the abstract after completing other sections to ensure it fully reflects the dissertation's content.
- Format in plain text (no HTML or special formatting) to comply with library archiving and online search requirements.

Keywords: database, HTML, library archiving, dissertation, methodology

致謝

首先，我想感謝周彤老師在這個 project 裡給予我許多幫助，不論從程式架構，以及方向上的建議，無疑都為整個研究帶給我可靠的支持。可以說，沒有他的協助，今天就不會有這篇論文的誕生。

接著，在這幾年的碩士生涯中，我的指導教授孫宏民老師提供給我一個很自由的實驗室環境，使我可以充分探索自己有興趣的題目。除此之外，孫老師也總是會確保實驗室擁有充足的研究資源，讓我在進行研究時毫無後顧之憂，很感謝他的幫忙。

最後我想感謝我的母親長期以來給我的關懷與支持，沒有她的照顧，今日我可能無法走到這一步，希望這一份小小的成就，可以讓她引以為傲。

Contents

摘要	i
Abstract	i
Acknowledgements	ii
Table of Contents	iv
List of Figures	v
List of Tables	vi
Nomenclature	1
1 Introduction	1
1.1 Motivation	1
1.2 Problem Statement	1
1.3 Contributions	1
2 Preliminaries	2
2.1 Notations	2
2.2 Quantum Computation	5
3 Greedy Algorithms for CNOT Gate Composition	6
3.1 Row/Col	6
3.2 Row-or-Col	6
3.3 Parallel	8
4 Cost Function	11
4.1 Log	11
4.2 Sum	11

4.3	Square	12
4.4	The Role of Matrix Cost Functions in Greedy Optimization	12
4.4.1	Globally	12
4.4.2	Locally	13
5	Chosen Improved Greedy Algorithm	17
5.1	Our Approach	17
5.2	Experiment Results	17
	References	21
	Appendices	
A	Codes	23
A.1	Content Elements	23
A.2	Writing Tips	23
B	Parameters	24
B.1	No. 0001 0050	24
B.2	No. 0051 0100	24

List of Figures

List of Tables

5.1	Results of the depth/gate-count of CNOT circuits for block ciphers	19
5.2	Comparison of the depth/gate-count of CNOT circuits for block ciphers .	20

Chapter 1

Introduction

1.1 Motivation

Quantum cryptography, as the name suggests, utilizes the cryptographic mechanisms underlying quantum circuits. To improve efficiency, it is essential to find more optimized implementations for these circuits. In quantum computing, many applications rely heavily on linear reversible circuits; therefore, deploying efficient methods to reduce their computational cost is key to providing more practical algorithms.

Currently, most symmetric crypto-systems are constructed using a significant number of CNOT gates in quantum circuit. Drawing inspiration from the field of Very Large Scale Integration (VLSI), we aim to reduce quantum algorithm costs by focusing on circuit depth as our primary metric, targeting linear scaling.

In this work, we propose an enhancement to the randomized greedy algorithm using heuristic methods to achieve the shallowest possible depth with the minimum number of CNOT gates. This approach not only guarantees a speedup in the execution of quantum cryptographic algorithms but also reduces the impact of quantum noise and hardware constraints on the computation results.

1.2 Problem Statement

Our research addresses the challenge of finding the optimal (minimum) depth for constructing the matrix components of symmetric ciphers in a quantum computing environment. All matrices are in $\mathbb{F}_2$.

1.3 Contributions

Chapter 2

Preliminaries

2.1 Notations

- M : Given an $n \times n$ input matrix A , our greedy algorithm seeks to identify a sequence of operations that transforms A into a permutation matrix.
- d : The circuit depth achieved when matrix A is transformed into a permutation matrix.
- M' : The matrix cost function is derived from the inverse of matrix A .
- SIZE: Since we are specifically considering square block matrices, the matrix size is determined by retrieving the row or column length using the built-in 'length()' function.
- L_r, L_c : Row and column operation sequences are stored in separate layer lists for each iteration.
- L_{s_r}, L_{s_c} : We maintain row-layer and column-layer lists to store the specific operations assigned to each discrete circuit layer.
- L_{row}, L_{col} : A list of remaining operations is updated after each step to exclude those already applied to the matrix.
- $L_{row}^{cost}, L_{col}^{cost}$: For every operation in L_{row} and L_{col} , the algorithm calculates and stores the resulting matrix cost function in a lookup list.
- row_{op}, col_{op} : The collection of applied row and column operations is stored to construct the final sequence list and to verify the correctness of the transformation.
- p : A control parameter is used to select the matrix cost function type. Supported

functions include linear sum (1), square (2), cubic (3), quartic (4), and logarithmic (- 1), which can be selected via their respective numerical identifiers.

- *over_depth*: An acceptance flag is utilized to evaluate the current circuit depth; it returns 'True' if the depth d exceeds the current minimum depth, and 'False' otherwise.
- *close_permu*: A termination flag monitors whether the matrix is nearing a permutation state. If the current matrix can be transformed into a permutation matrix within a single layer, the flag is set to True; otherwise, it remains False.
- *minm*: The algorithm identifies the current minimum cost across all candidates and selects the operation corresponding to this optimal value.
- *origin, inverse*: At the start of the process, a copy of the input matrix is retained for final verification. Simultaneously, the matrix inverse is calculated to facilitate the evaluation of the cost function.
- *visi_{row}, visi_{col}*: A visited list tracks the selected row and column indices; once a position is chosen, it is excluded from future iterations to prevent redundant operations.
- *select_list*: The selection list stores the set of candidate operations available for the current optimization step.
- *H_r, H_c*: The matrix cost function is evaluated from both a row perspective and a column perspective to determine the most efficient operation.
- *B_{row}, B_{col}*: The algorithm maintains a collection of row and column operations for both the Row/Columnwise and Parallel greedy approaches; it identifies the available operators and selects the one with the lowest computational cost.
- *best_{row}^{cost}, best_{col}^{cost}*: The algorithm maintains a collection of cost values for each potential operation in B_{row} and B_{col} to facilitate the selection of the optimal operator.
- *stuck_counter*: A counter is used to record the number of consecutive stalls (stuck

iterations) to detect when the algorithm has reached a local optimum.

- `max_stuck_iteration`: A stagnation threshold is defined to limit the number of consecutive stalls; by default, this value is set to 3.
- `escapeCandidates`: The algorithm also maintains a collection of non-optimal operators, representing candidates whose cost values exceeded the current minimum.
- `best_escape`: To resolve local minima, the algorithm identifies the optimal candidate from the `escapeCandidates` list—after sorting it in ascending order—and adds it to the `select_list` to resume the search.
- `occur_time`: The acceptance counter records the total number of iterations in which an operation was successfully selected and applied.
- `minm_size`: A threshold condition is implemented to ensure that the circuit depth (total number of layers) does not exceed the predefined limit.
- `reduce`: A copy of the initial matrix is retained to verify that the sequence of row and column operations (row_{op} and col_{op}) successfully transforms the origin matrix into the target representation.
- `size_minm`: The algorithm records the final circuit depth to document the efficiency of the optimized sequence.
- `per`: The algorithm records the final matrix state to generate a permutation list, which defines the final mapping of the quantum registers.
- `seq`: The algorithm maintains a comprehensive log of all applied row and column operations (row_{op} and col_{op}), which constitutes the final quantum circuit sequence.
- `layers`: The algorithm maintains a record of the sets LS_c and LS_r , which contain the column and row operations assigned to each parallel layer.
- `correct`: A validation flag is employed to confirm that the combined operation sequence and layer assignments successfully transform the original matrix into a permutation matrix.

- greedy: An algorithmic selection parameter allows the user to specify which greedy strategy (e.g., Row, Column, Row_or_Col or Parallel) should be employed during the synthesis process.

2.2 Quantum Computation

Chapter 3

Greedy Algorithms for CNOT Gate Composition

In our work, we adopt the heuristic method to find the most shallow depth for CNOT gate, after we survey on

Building on the work of [2], who proposed an improved depth-oriented algorithm for synthesizing ancilla-free, low-depth CNOT circuits, we developed four distinct greedy strategies to further explore depth-optimized results.

By utilizing different matrix representations and search heuristics, we implemented the Row, Column, Row-or-Column, and Parallel algorithms.

The pseudocode and implementation logic for each approach are detailed in the following sections.

3.1 Row/Col

This strategy aims to drive the matrix toward the 'can_one' condition. Once the 'one' flag is triggered, the algorithm shifts its priority exclusively to row operations to facilitate the final reduction into a permutation matrix.

3.2 Row-or-Col

This strategy evaluates both row and column operations and selects the layer that yields the minimum matrix cost. In cases where the costs are identical, the algorithm defaults to the column-wise operation.

If only one set of operations is available, the algorithm selects the existing set.

Algorithm 1 Row/Col Greedy

Input: $M \in \mathbb{F}_2^{n \times n}$, $M' \in \mathbb{F}_2^{n \times n}$, $L_r, L_c, L_{s_r}, L_{s_c}, p$ and `over_depth`

Output: $L_{s_r}, L_{s_c}, row_{op}, col_{op}$ that implement permutation matrix with length d

```
1: SIZE  $\leftarrow$  length( $M$ )
2: minm  $\leftarrow$  system.float_info.max
3: select_list  $\leftarrow$  list()
4:  $visi_{row} \leftarrow [\emptyset]$ ,  $visi_{col} \leftarrow [\emptyset]$ 
5: close_permu  $\leftarrow$  False
6: LIMIT  $\leftarrow$  LATEST_MINM_DEPTH
7: while  $M \neq$  Permutation do
8:   select_list.clear()
9:    $L_{row}.clear()$ ,  $L_{col}.clear()$ ,  $L_{row}^{cost}.clear()$ ,  $L_{col}^{cost}.clear()$ 
10:   $H_r, H_c$ , minm  $\leftarrow$  functionCost( $M, M', p$ )
11:  if !close_permu then
12:     $L_{col} \leftarrow L\_Collection(visi_{row})$ 
13:    select_list, minm  $\leftarrow$  Row_Op_Sel( $L_{col}, L_{col}^{cost}, M, M', p$ )
14:     $L_{row} \leftarrow L\_Collection(visi_{row})$ 
15:    function AVAILABLEROWCOLLECTION( $visi_{row}$ , SIZE)
16:    | return  $L_{row}$ 
17:    function AVAILABLEROWOPERATORSELECTION( $L_{row}, L_{row}^{cost}, mat, inv, p$ )
18:    | return select_list, minm
19:    function AVAILABLEOPERATOREXECUTION(select_list,  $L_r, L_c, L_{s_r}, L_{s_c}, mat$ ,
20:     $inv, row_{op}, col_{op}, visi_{row}, visi_{col}, d$ , SIZE, one)
21:    | return  $L_r, L_c, L_{s_r}, L_{s_c}, mat, inv, row_{op}, col_{op}, visi_{row}, visi_{col}, d$ , one
22:    if  $d \geq$  LIMIT then
23:    |  $flag \leftarrow$  True
24:    | return  $L_r, L_c, L_{s_r}, L_{s_c}, mat, row_{op}, col_{op}, d$ , flag
```

To address the inherent risk of local minima in greedy heuristics, we implement a stuck threshold. If the cost fails to decrease for three consecutive iterations, the algorithm accepts a sub-optimal operation that increases the cost, thereby enabling an escape from the local minimum.

3.3 Parallel

The Parallel Greedy strategy aggregates both row and column operations into a unified ‘select_list’.

In cases where one operation type is unavailable, the algorithm proceeds with the existing candidates. Similar to the Row-or-Column approach, a stagnation threshold is utilized to escape local minima; if no progress is made for three iterations, the algorithm accepts cost-increasing moves.

The primary distinction of the Parallel strategy lies in its late-stage optimization: when the matrix approaches a permutation state, the ‘one’ flag is activated, restricting the search to column-wise operations to finalize the circuit.

Algorithm 2 Row_or_Col Greedy

Input: $mat_{n \times n}, inv_{n \times n}, L_r, L_c, L_{s_r}, L_{s_c}, p$ and $flag$

Output: $L_{s_r}, L_{s_c}, row_{op}, col_{op}$ that implement permutation matrix with length d

```
1: SIZE  $\leftarrow$  length(mat)
2: minm,  $best_{row}^{cost}, best_{col}^{cost} \leftarrow$  system.float_info.max
3: select_list  $\leftarrow$  list()
4:  $visi_{row} \leftarrow [0], visi_{col} \leftarrow [0]$ 
5: LIMIT  $\leftarrow$  LATEST_MINM_DEPTH
6: stuck_counter  $\leftarrow$  0, max_stuck_iterations  $\leftarrow$  3
7: while  $mat \neq$  Permutation do
8:   select_list.clear()
9:    $L_{row}.clear(), L_{col}.clear(), L_{row}^{cost}.clear(), L_{col}^{cost}.clear(), B_{row}.clear(), B_{col}.clear()$ 
10:  if  $p \neq 1$  then
11:     $H_r \leftarrow \text{costMat}(mat, p) + \text{costMat}(inv^T, p)$ 
12:     $H_c \leftarrow \text{costMat}(mat^T, p) + \text{costMat}(inv, p)$ 
13:    minm  $\leftarrow$  max( $H_r, H_c$ )
14:  else
15:    minm  $\leftarrow$  costMat( $mat, p$ ) + costMat( $inv, p$ )
16:  function AVAILABLEROWCOLLECTION( $visi_{row}, SIZE$ )
17:    return  $L_{row}$ 
18:  function MODIFIEDAVAILABLEROWOPERATORSELECTION( $L_{row}, L_{row}^{cost}, mat, inv, p, minm$ )
19:    return  $B_{row}, best_{row}^{cost}$ 
20:  function AVAILABLECOLCOLLECTION( $visi_{col}, SIZE$ )
21:    return  $L_{col}$ 
22:  function MODIFIEDAVAILABLECOLOPERATORSELECTION( $L_{col}, L_{col}^{cost}, mat, inv, p, minm$ )
23:    return  $B_{col}, best_{col}^{cost}$ 
24:  is_stuck = length( $B_{row}$ ) == 0  $\wedge$  length( $B_{col}$ ) == 0
25:  if is_stuck then
26:    stuck_counter += 1
27:  else
28:    stuck_counter  $\leftarrow$  0
29:  if (stuck_counter  $\geq$  max_stuck_iterations) OR (is_stuck  $\wedge \sum visi_{row} == 0 \wedge \sum visi_{col} == 0$ ) then
30:    escapeCandidates.clear()
31:    function AVOIDLOCALMINIMAAVAILABLEOPERATORSELECTION( $L_{row}, L_{col}, mat, inv, p,$ 
    escapeCandidates)
32:    return select_list, minm
33:  if  $best_{row}^{cost} < best_{col}^{cost}$  then
34:    select_list  $\leftarrow B_{row}$ 
35:    minm  $\leftarrow best_{row}^{cost}$ 
36:  else if  $best_{row}^{cost} > best_{col}^{cost}$  then
37:    select_list  $\leftarrow B_{col}$ 
38:    minm  $\leftarrow best_{col}^{cost}$ 
39:  else
40:    select_list  $\leftarrow B_{col}$  if len( $B_{col}$ ) > 0 else  $B_{row}$ 
41:    minm  $\leftarrow best_{col}^{cost}$  if len(select_list) > 0 else minm
42:  function AVAILABLEOPERATOREXECUTION(select_list,  $L_r, L_c, L_{s_r}, L_{s_c}, mat, inv,$ 
     $row_{op}, col_{op}, visi_{row}, visi_{col}, d, SIZE$ )
43:    return  $L_r, L_c, L_{s_r}, L_{s_c}, mat, inv, row_{op}, col_{op}, visi_{row}, visi_{col}, d$ 
44:  if  $d \geq$  LIMIT then
45:    flag = True
46:  return
47: return  $L_r, L_c, L_{s_r}, L_{s_c}, mat, row_{op}, col_{op}, d, flag$ 
```

Algorithm 3 Parallel Greedy

Input: $mat_{n \times n}, inv_{n \times n}, L_r, L_c, Ls_r, Ls_c, p$ and $flag$

Output: $Ls_r, Ls_c, row_{op}, col_{op}$ that implement permutation matrix with length d

```
1: SIZE  $\leftarrow$  length(mat)
2: minm,  $best_{row}^{cost}, best_{col}^{cost} \leftarrow$  system.float_info.max
3: select_list  $\leftarrow$  list()
4:  $visi_{row} \leftarrow [0], visi_{col} \leftarrow [0]$ 
5: LIMIT  $\leftarrow$  LATEST_MINM_DEPTH
6: one  $\leftarrow$  False
7: stuck_counter  $\leftarrow$  0, max_stuck_iterations  $\leftarrow$  3
8: while  $mat \neq$  Permutation do
9:   select_list.clear()
10:   $L_{row}.clear(), L_{col}.clear(), L_{row}^{cost}.clear(), L_{col}^{cost}.clear(), B_{row}.clear(), B_{col}.clear()$ 
11:  if  $p \neq 1$  then
12:     $H_r = \text{costMat}(mat, p) + \text{costMat}(inv^T, p)$ 
13:     $H_c = \text{costMat}(mat^T, p) + \text{costMat}(inv, p)$ 
14:    minm = max( $H_r, H_c$ )
15:  else
16:    minm = costMat( $mat, p$ ) + costMat( $inv, p$ )
17:  if !one then
18:    function AVAILABLEROWCOLLECTION( $visi_{row}, SIZE$ )
19:    | return  $L_{row}$ 
20:    function MODIFIEDAVAILABLEROWOPERATORSELECTION( $L_{row}, L_{row}^{cost}, mat, inv, p, minm$ )
21:    | return  $B_{row}, best_{row}^{cost}$ 
22:    function AVAILABLECOLLECTION( $visi_{col}, SIZE$ )
23:    | return  $L_{col}$ 
24:    function MODIFIEDAVAILABLECOLOPERATORSELECTION( $L_{col}, L_{col}^{cost}, mat, inv, p, minm$ )
25:    | return  $B_{col}, best_{col}^{cost}$ 
26:  is_stuck = length( $B_{row}$ ) == 0  $\wedge$  length( $B_{col}$ ) == 0
27:  if is_stuck then
28:    stuck_counter += 1
29:  else
30:    stuck_counter  $\leftarrow$  0
31:  if (stuck_counter  $\geq$  max_stuck_iterations) OR (is_stuck  $\wedge \sum visi_{row} == 0 \wedge \sum visi_{col} == 0$ ) then
32:    escapeCandidates.clear()
33:    function AVOIDLOCALMINIMAAVAILABLEOPERATORSELECTION( $L_{row}, L_{col}, mat, inv, p,$ 
34:    | return select_list, minm
35:  else
36:    select_list  $\leftarrow B_{row} + B_{col}$ 
37:    if  $best_{row}^{cost} < best_{col}^{cost}$  then
38:    | minm  $\leftarrow best_{row}^{cost}$ 
39:    else if  $best_{row}^{cost} > best_{col}^{cost}$  then
40:    | minm  $\leftarrow best_{col}^{cost}$ 
41:    else
42:    | minm  $\leftarrow best_{col}^{cost}$  if len(select_list) > 0 else minm
43:  function AVAILABLEOPERATOREXECUTION(select_list,  $L_r, L_c, Ls_r, Ls_c, mat, inv,$ 
44:  | return  $L_r, L_c, Ls_r, Ls_c, mat, inv, row_{op}, col_{op}, visi_{row}, visi_{col}, d, one$ 
45:  if  $d \nless$  LIMIT then
46:    flag  $\leftarrow$  True
47:  return
48: return  $L_r, L_c, Ls_r, Ls_c, mat, row_{op}, col_{op}, d, flag$ 
```

Chapter 4

Cost Function

The ‘costMat’ function serves as our primary heuristic, allowing for the selection of five distinct cost metrics. These metrics—logarithmic, linear sum, square, cubic, and quartic—are controlled by a parameter p , which determines the mathematical weight assigned to the matrix reduction process.

4.1 Log

This cost function is derived from the greedy framework proposed by [1], which utilizes the logarithm of the Hamming weight for each row as follows:

$$h_{log}(A) = \sum_i \log_2(\sum_j a_{i,j})$$

4.2 Sum

In addition to the logarithmic metric, the greedy framework by [1] considers the cumulative Hamming weight of each row as follows:

$$h_{sum}(A) = \sum_i \sum_j a_{i,j}$$

4.3 Square

Following the methodology of [2], this cost function utilizes the square of each row's Hamming weight, defined as follows:

$$h_{sq}(A) = \sum_i (\sum_j a_{i,j})^2$$

4.4 The Role of Matrix Cost Functions in Greedy Optimization

The ‘costMat’ function is integrated into two distinct phases of our greedy algorithm: the initial candidate selection and the subsequent layer evaluation.

4.4.1 Globally

First, the algorithm evaluates the global maximum cost to establish a relaxation range; this allows the search to include a broader set of available operators rather than being restricted to the absolute minimum.

Algorithm 4 Greedy

```

1:  $\vdots$ 
2: while  $mat \neq \text{Permutation}$  do
3:    $\vdots$ 
4:   if  $p \neq 1$  then
5:      $H_r \leftarrow \text{costMat}(mat, p) + \text{costMat}(inv^T, p)$ 
6:      $H_c \leftarrow \text{costMat}(mat^T, p) + \text{costMat}(inv, p)$ 
7:      $\text{minm} \leftarrow \max\{H_r, H_c\}$ 
8:   else
9:      $\text{minm} \leftarrow \text{costMat}(mat, p) + \text{costMat}(inv, p)$ 
10:   $\vdots$ 

```

Following the greedy framework established by Shi and Feng, we define the cost metrics H_r and H_c to treat row and column operations asymmetrically. Specifically, the row Hamming weight is evaluated when performing row operations (row_{op}), while the

column Hamming weight is prioritized during column operations (col_{op}).

To explore the set of linear reversible operators for synthesizing quantum cryptographic circuits, we employ the following guidance:

$$H_r = costMat(A, p) + costMat((A^{-1})^T, p)$$

$$H_c = costMat(A^T, p) + costMat(A^{-1}, p)$$

4.4.2 Locally

In the second phase, the algorithm performs a local evaluation to identify individual operators that minimize the cost function. If a candidate operator's cost is lower than the current minimum, it is accepted as a valid transformation.

Once all potential control and target qubit pairs have been visited, the resulting set of parallel operations is appended to the 'select_list' as a single circuit layer.

The availableRowOperatorSelection and availableColOperatorSelection functions influence the cost matrix calculation in the Row and Col greedy algorithms.

Algorithm 5 availableRowOperatorSelection

Input: L_{row} , L_{row}^{cost} , $mat_{n \times n}$, $inv_{n \times n}$, p , $minm$ and $select_list$

Output: $select_list$, $minm$

```

1: for  $row_{op} \in L_{row}$  do
2:    $temp\_mat \leftarrow row\_i2j(mat, row_{op}[0], row_{op}[1])$ 
3:    $temp\_inv \leftarrow col\_i2j(inv, row_{op}[1], row_{op}[0])$ 
4:   if  $p \neq 1$  then
5:      $L_{row}^{cost}.append(costMat(temp\_mat, p) + costMat(temp\_inv^T, p))$ 
6:   else
7:      $L_{row}^{cost}.append(costMat(temp\_mat, p) + costMat(temp\_inv, p))$ 
8:   for  $index, row_{op}^{cost}$  in  $L_{row}^{cost}$  do
9:     if  $row_{op}^{cost} < minm$  then
10:       $select\_list.clear()$ 
11:       $select\_list.append((L_{row}[index][0], L_{row}[index][1], 0))$ 
12:       $minm = row_{op}^{cost}$ 
13:     else if  $row_{op}^{cost} == minm$  then
14:        $select\_list.append((L_{row}[index][0], L_{row}[index][1], 0))$ 
15: return  $select\_list, minm$ 

```

Algorithm 6 availableColOperatorSelection

Input: L_{col} , L_{col}^{cost} , $mat_{n \times n}$, $inv_{n \times n}$, p , $minm$ and $select_list$

Output: $select_list$, $minm$

```
1: for  $col_{op} \in L_{col}$  do
2:    $temp\_mat \leftarrow col\_i2j(mat, col_{op}[0], col_{op}[1])$ 
3:    $temp\_inv \leftarrow row\_i2j(inv, col_{op}[1], col_{op}[0])$ 
4:   if  $p \neq 1$  then
5:      $L_{col}^{cost}.append(costMat(temp\_mat^T, p) + costMat(temp\_inv, p))$ 
6:   else
7:      $L_{col}^{cost}.append(costMat(temp\_mat, p) + costMat(temp\_inv, p))$ 
8:   for  $index, col_{op}^{cost}$  in  $L_{col}^{cost}$  do
9:     if  $col_{op}^{cost} < minm$  then
10:       $select\_list.clear()$ 
11:       $select\_list.append((L_{col}[index][0], L_{col}[index][1], 1))$ 
12:       $minm = col_{op}^{cost}$ 
13:     else if  $col_{op}^{cost} == minm$  then
14:        $select\_list.append((L_{col}[index][0], L_{col}[index][1], 1))$ 
15: return  $select\_list, minm$ 
```

Similarly, the modified and avoidLocalMinima versions of these functions influence the cost matrix in the Row-or-Col and Parallel greedy algorithms.

For the Row-or-Column and Parallel strategies, which evaluate two directional operators simultaneously, the algorithm selects the layer that minimizes the total matrix cost.

To facilitate this, we implement an intermediate storage list for candidate operations and utilize their respective cost impacts as the primary selection criterion.

This represents a departure from standard operator selection, as detailed in the following modifications:

To address the issue of local minima, we implemented a stagnation recovery mechanism. When the algorithm becomes 'stuck'—meaning no cost-reducing operations are found—the greedy constraint is temporarily relaxed.

During these instances, the algorithm accepts operations that slightly increase the current minimum cost.

This heuristic perturbation enables the search to bypass stagnant states and discover subsequent operations that lead to a more optimal global reduction.

Algorithm 7 modifiedAvailableRowOperatorSelection

Input: $L_{row}, L_{row}^{cost}, mat_{n \times n}, inv_{n \times n}, p, minm, best_{row}^{cost}$ and B_{row}

Output: $B_{row}, best_{row}^{cost}, minm$

```
1: for  $row_{op} \in L_{row}$  do
2:   temp_mat  $\leftarrow$  row_i2j(mat, row_op[0], row_op[1])
3:   temp_inv  $\leftarrow$  col_i2j(inv, row_op[1], row_op[0])
4:   if  $p \neq 1$  then
5:      $L_{row}^{cost}.append(costMat(temp\_mat, p) + costMat(temp\_inv^T, p))$ 
6:   else
7:      $L_{row}^{cost}.append(costMat(temp\_mat, p) + costMat(temp\_inv, p))$ 
8: for index,  $row_{op}^{cost}$  in  $L_{row}^{cost}$  do
9:   if  $row_{op}^{cost} < minm$  then
10:    if  $row_{op}^{cost} < best_{row}^{cost}$  then
11:       $B_{row}.clear()$ 
12:       $best_{row}^{cost} = row_{op}^{cost}$ 
13:      if  $row_{op}^{cost} == best_{row}^{cost}$  then
14:         $B_{row}.append((L_{row}[index][0], L_{row}[index][1], 0))$ 
15: return  $B_{row}, best_{row}^{cost}$ 
```

Algorithm 8 modifiedAvailableColOperatorSelection

Input: $L_{col}, L_{col}^{cost}, mat_{n \times n}, inv_{n \times n}, p, minm, best_{col}^{cost}$ and B_{col}

Output: $B_{col}, best_{col}^{cost}, minm$

```
1: for  $col_{op} \in L_{col}$  do
2:   temp_mat  $\leftarrow$  col_i2j(mat, col_op[0], col_op[1])
3:   temp_inv  $\leftarrow$  row_i2j(inv, col_op[1], col_op[0])
4:   if  $p \neq 1$  then
5:      $L_{col}^{cost}.append(costMat(temp\_mat^T, p) + costMat(temp\_inv, p))$ 
6:   else
7:      $L_{col}^{cost}.append(costMat(temp\_mat, p) + costMat(temp\_inv, p))$ 
8: for index,  $col_{op}^{cost}$  in  $L_{col}^{cost}$  do
9:   if  $col_{op}^{cost} < minm$  then
10:    if  $col_{op}^{cost} < best_{col}^{cost}$  then
11:       $B_{col}.clear()$ 
12:       $best_{col}^{cost} = col_{op}^{cost}$ 
13:      if  $col_{op}^{cost} == best_{col}^{cost}$  then
14:         $B_{col}.append((L_{col}[index][0], L_{col}[index][1], 1))$ 
15: return  $B_{col}, best_{col}^{cost}$ 
```

Algorithm 9 avoidLocalMinimaRowOperatorSelection

Input: L_{row} , $mat_{n \times n}$, $inv_{n \times n}$, p and escapeCandidates

Output: escapeCandidates

```
1: for  $row_{op} \in L_{row}$  do
2:    $temp\_mat \leftarrow row\_i2j(mat, row_{op}[0], row_{op}[1])$ 
3:    $temp\_inv \leftarrow col\_i2j(inv, row_{op}[1], row_{op}[0])$ 
4:   if  $p \neq 1$  then
5:      $temp\_cost \leftarrow costMat(temp\_mat, p) + costMat(temp\_inv^T, p)$ 
6:   else
7:      $temp\_cost \leftarrow costMat(temp\_mat, p) + costMat(temp\_inv, p)$ 
8:    $escapeCandidates.append((temp\_cost, row_{op}[0], row_{op}[1], 0))$ 
9: return escapeCandidates
```

Algorithm 10 avoidLocalMinimaColOperatorSelection

Input: L_{col} , $mat_{n \times n}$, $inv_{n \times n}$, p and escapeCandidates

Output: escapeCandidates

```
1: for  $col_{op} \in L_{col}$  do
2:    $temp\_mat \leftarrow col\_i2j(mat, col_{op}[0], col_{op}[1])$ 
3:    $temp\_inv \leftarrow row\_i2j(inv, col_{op}[1], col_{op}[0])$ 
4:   if  $p \neq 1$  then
5:      $temp\_cost \leftarrow costMat(temp\_mat^T, p) + costMat(temp\_inv, p)$ 
6:   else
7:      $temp\_cost \leftarrow costMat(temp\_mat, p) + costMat(temp\_inv, p)$ 
8:    $escapeCandidates.append((temp\_cost, col_{op}[0], col_{op}[1], 1))$ 
9: return escapeCandidates
```

Chapter 5

Chosen Improved Greedy Algorithm

5.1 Our Approach

The primary algorithm integrating these cost metrics and greedy heuristics is detailed below:

5.2 Experiment Results

Algorithm 11 Chosen Improved Greedy Algorithm

Input: *mat*, *matName*, *greedy*, *p*, *occur_time*

Output: *d*, *Layers*

```
1: minm, minm_size  $\leftarrow$  system.float_info.max
2: SIZE  $\leftarrow$  len(mat)
3: origin.copy(mat)
4: inv  $\leftarrow$  inverse(mat)
5: Lr, Lc, Lsr, Lsc, rowop, colop  $\leftarrow$  list()
6: flag  $\leftarrow$  False
7: function MODIFIEDRANDOMIZEDGREEDY(greedy, p)
8: | return Lr, Lc, Lsr, Lsc, mat, rowop, colop, depth, flag
9: if flag then
10: | return
11: function LASTCHECK(Lr, Lc, Lsr, Lsc)
12: | return Lsr, Lsc
13: reduce.copy(origin)
14: size  $\leftarrow$  0
15: function GENERATEREDUCEMATRIX(reduce, rowop, colop, size)
16: | return reduce, size
17: ok  $\leftarrow$  True
18: if reduce  $\neq$  mat then
19: | ok = False
20: if (d > minm) OR (d == minm  $\wedge$  size  $\geq$  minm_size) OR !ok then
21: | return
22: minm  $\leftarrow$  d
23: size_minm  $\leftarrow$  size
24: per  $\leftarrow$  list()
25: function GENERATEPERMUTATIONLIST(SIZE, mat, per)
26: | return per
27: seq  $\leftarrow$  list()
28: function GENERATESEQUENCELIST(seq, rowop, colop)
29: | return seq
30: layers  $\leftarrow$  list()
31: function GENERATELAYERSLIST(layers, Lsr, Lsc)
32: | return layers
33: correct  $\leftarrow$  Verify(origin, layers, seq, mat)
34: if !correct then
35: | return
36: return d, layers
```

Cipher	Cost Function	Row	Column	Row-or-Col	Parallel
AES	Log	32	30/92	10/131	10/129
	Sum				
	Square				
	Origin				
FOX MU4	Log	32	30/92	10/131	10/129
	Sum				
	Square				
	Origin				
TWOFISH	Log	32	30/92	10/131	10/129
	Sum				
	Square				
	Origin				
WHIRLWIND M0	Log	32	30/92	10/131	10/129
	Sum				
	Square				
	Origin				
WHIRLWIND M1	Log	32	30/92	10/131	10/129
	Sum				
	Square				
	Origin				

Table 5.1: Results of the depth/gate-count of CNOT circuits for block ciphers

Cipher	Size	Q#	[2]	Our Algorithm
AES	32	30/92	10/131	
FOX MU4	32	55/136	21/265	
TWOFISH	32	37/111	15/175	
WHIRLWIND M0	32	65/183	28/331	
WHIRLWIND M1	32	69/190	22/290	

Table 5.2: Comparison of the depth/gate-count of CNOT circuits for block ciphers

References

- [1] Timothée Goubault de Brugière et al. “Reducing the Depth of Linear Reversible Quantum Circuits”. In: *IEEE Transactions on Quantum Engineering* 2 (2021), pp. 1–22. DOI: [10.1109/TQE.2021.3091648](https://doi.org/10.1109/TQE.2021.3091648).
- [2] Haotian Shi and Xiutao Feng. *Quantum Circuits of AES with a Low-depth Linear Layer and a New Structure*. Cryptology ePrint Archive, Paper 2024/381. 2024. URL: <https://eprint.iacr.org/2024/381>.

Appendices

Appendix A

Codes

Appendices include supplementary materials that enhance the dissertation's comprehensiveness without disrupting the main text's flow.

A.1 Content Elements

- Include detailed datasets, code, raw measurements, additional figures, derivations, or equipment schematics.
- Number and provide a brief description for each appendix, linking it to the main text.

A.2 Writing Tips

- Include only materials that aid understanding but are too lengthy for the main text.
- Ensure appendices follow the same formatting standards as the main document.
- De-identify data, especially for human subjects or sensitive information, to comply with ethical standards.

Appendix B

Parameters

B.1 No. 0001 0050

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

B.2 No. 0051 0100

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.